

NATIONAL UNIVERSITY OF HO CHI MINH CITY
UNIVERSITY OF INFORMATION TECHNOLOGY
FACULTY OF COMPUTER ENGINEERING

CIRCUIT DESIGN WITH HDL

CHAPTER 6: TASKS & FUNCTIONS

Lecturer: Ho Ngoc Diem

Course outline

- Chapter 1: Introduction
- Chapter 2: Verilog Language Concepts
- Chapter 3: Structural modeling
- Chapter 4: Behavioral modeling
- Chapter 5: Finite State machines
- **Chapter 6: Tasks and Functions**
- Chapter 7: Functional Simulation/Verification (Testbench)
- Chapter 8: Synthesis of Combinational and Sequential Logic
- Chapter 9: Post-synthesis design tasks
- Chapter 10: VHDL introduction

Introduction

- Procedures/Subroutines/Functions in SW programming languages
 - The same functionality, in different places
- Verilog equivalence:
 - *Tasks and Functions*
 - Used in Behavioral Modeling
 - Provide the ability to execute common procedures from several different places in a description.

Contents

- Definition of Functions
- Definition of Tasks
- Differences between Tasks and Functions

Functions

- **Functions** are defined *in the module* in which they are used.

It is possible to define functions in separate files and use compile directive *'include* to include the function in the file which instantiates one.

- The purpose of a **function** is to return a value that is to be used in an expression
- **Functions** *can not include timing delays*, like *@*, *# delay*, *wait* which means that functions should be executed in "zero" time delay.
- **Functions** can have *any number of inputs* but can not have *output* or *inout* argument.

Functions

- The variables declared within the **function** are local to that function.
 - *The order of declaration within the function defines how the variables passed to the function by the caller are used.*
- **Functions** can take, drive, and source global variables, when no local variables are used.

When local variables are used, basically output is assigned only at the end of function execution.

- **Functions** can be used for **modeling combinational logic**.
- **Functions** can **call other functions**, but **can not call tasks**.

Functions

- Keyword: `function`, `endfunction`
- Can be used if the procedure
 - does not have any timing control constructs
 - returns exactly a single value
 - has at least one input argument

Functions

- Function Declaration and Invocation
 - Declaration syntax:

Must be written inside a module declaration

```
function <range_or_type> func_name (<func_port_list>);  
    <input_declaration(s)> //either <func_port_list>  
                           //or <input_declaration> is used  
    <variable_declaration(s)>  
  
    begin // if more than one statement needed  
        <statements>  
  
    end  
  
endfunction
```

Functions

- Semantics
 - much like `function` in *Pascal*
 - An internal implicit `variable` is declared inside the function with the same name
 - The return value is specified by setting that implicit to a default `1-bit reg` or the same type as the type specified in the function
 - declaration. `<range_or_type>` defines width and type of the implicit `variable`
 - `<type>` can be `integer`, `real`, `time`, `realtime`
 - **default bit width is 1**

Functions

- Function Declaration and Invocation

- Invocation syntax:

`func_name (<argument(s)>) ;`

- The same function can be used more than once within the same module.

Function Example - Parity Generator

```
module parity;
reg [31:0] addr;
reg parity;

initial begin
    ...
end

always @(addr)
begin
    parity = calc_parity(addr);
    $display("Parity calculated =
    %b", calc_parity(addr) );
end
```

Reg 1 bit

```
function calc_parity;
input [31:0] address;
begin
    calc_parity = ^address;
end
endfunction

endmodule
```

Same name

The implicit reg 1 bit width as default

Function Example - Controllable Shifter

```
module shifter;
`define LEFT_SHIFT      1'b0
`define RIGHT_SHIFT    1'b1
reg [31:0] addr, left_addr,
           right_addr;
reg control;

initial
begin
    .....
end

always @(addr)
begin
    left_addr  =shift(addr, `LEFT_SHIFT);
    right_addr =shift(addr, `RIGHT_SHIFT);
end
```

```
function [31:0] shift;
input [31:0] address;
input control;
begin
    shift =
        (control==`LEFT_SHIFT) ?
            (address<<1) :
            (address>>1);
end
endfunction

endmodule
```

“32 bit width”

Function Example – calling a function coded in a separate file

```
module  function_calling
        (a, b, c, d, e, f);

input a, b, c, d, e ;
output f;
wire f;
`include "myfunction.txt"

assign f =
    (myfunction (a,b,c,d)) ? e :0;

endmodule
```

```
function  myfunction;
input a, b, c, d;
begin
    myfunction = ((a+b) + (c-d));
end
endfunction
```

Function is written in a separate file
→ it can be used in multiple modules

Source: http://www.asic-world.com/verilog/task_func1.html#Task

Function rules - remember

A function:

- shall not contain any time-controlled statements (`#`, `@`, or `wait`);
 - shall not enable tasks;
 - shall contain at least one input argument
 - shall not have any argument declared as output or inout
 - shall not have any nonblocking assignments or procedural continuous assignments;
 - shall not have any event triggers
- **Functions are always combinatorial**, so there is no doubt about what type of logic is being modeled.

Tasks

- **Tasks** are used in all programming languages, generally known as procedures or subroutines. The lines of code are enclosed in `task....endtask`.
- Data is passed to the **task**, the processing done, and the result returned.
- Included in the main body of code, **tasks** can be called many times, *reducing code repetition*.

Source: http://www.asic-world.com/verilog/task_func1.html#Task

Tasks

- **Tasks** are defined in the module in which they are used.
 - It is possible to define a **task** in a separate file and use the compile directive *'include* to include the task in the file which instantiates the task.
- **Tasks** can include timing delays, like @, # delay and wait.
- **Tasks** can have any number of inputs and outputs. The variables declared within the task are local to that task.
 - *The order of declaration within the task defines how the variables passed to the task by the caller are used.*

Source: http://www.asic-world.com/verilog/task_func1.html#Task

Tasks

- **Tasks** can take, drive and source global variables, when no local variables are used.
 - When local variables are used, basically output is assigned only at the end of task execution.
- **Tasks** *can call another task or function*.
- **Tasks** can be used for modeling both combinational and sequential logic.
- A **Task** must be specifically called with a statement, it *cannot be used within an expression* as a function can.

Source: http://www.asic-world.com/verilog/task_func1.html#Task

Tasks

- Keywords: `task`, `endtask`
- A task can support multiple goals and can calculate multiple result values.
- *Inputs* and *outputs* are declared after the keyword `task`.
- *Local variables* are declared after input and output declaration.
- Only the output or inout type arguments pass result values back from the invocation of a task.

Tasks

- Task declaration
 - Declaration syntax

Must be written inside a
module declaration

```
task task_name (<task_port_list>;  
    <I/O declarations> // either (<task_port_list>) or  
                        //<I/O declaration> is used  
    <variable and event declarations>  
begin // if more than one statement needed  
    <statement(s)>  
end // if begin used  
endtask
```

Tasks

- Task invocation

- Invocation syntax

- `task_name;`

- `task_name (<arguments>);`

- `input` and `inout` arguments are passed into the task
 - `output` and `inout` arguments are passed back to the invoking statement when task is completed

Tasks

- I/O declaration in **module** vs. **task**
 - Both used keywords: `input`, `output`, `inout`
 - **In modules**, represent ports
 - connect to external signals
 - **In tasks**, represent arguments
 - pass values to and from the task

Task example -

Using Task global variables

```
module task_global();  
  
    reg [7:0] temp_out;  
    reg [7:0] temp_in;  
  
    task convert;  
    begin  
        temp_out = (9/5) *( temp_in + 32);  
    end  
endtask  
  
endmodule
```

Source: http://www.asic-world.com/verilog/task_func1.html#Task

Task example -

Usage of Module Local/Task Global Variables

```
module sequence;
  reg clock;

  initial
  begin
    <main>
  end

  initial
    init_sequence;

  always
    asymmetric_sequence;
```

```
task init_sequence;
    clock = 1'b0;
endtask

task asymmetric_sequence;
begin
    #12 clock = 1'b0;
    #5  clock = 1'b1;
    #3  clock = 1'b0;
    #10 clock = 1'b1;

end
endtask

endmodule
```

Task example -

Usages of Input and Output arguments

```
module operation;
parameter delay = 10;
reg [15:0] A, B;
reg [15:0] AB_AND, AB_OR, B_XOR;

initial
    $monitor( ... );

initial
begin
    ...
end

always @(A or B)
begin
    bitwise_oper(AB_AND, AB_OR,
                AB_XOR, A, B);
end
```

```
task bitwise_oper;
output [15:0] ab_and, ab_or,
           ab_xor;
input [15:0] a, b;
begin
    #delay ab_and = a & b;
    ab_or = a | b;
    ab_xor = a ^ b;
end
endtask

endmodule
```

Note: the order of Outs/Ins in Task

Task example 1-

Calling a task coded in a separate file

```
module task_calling (temp_a, temp_b,  
                    temp_c, temp_d);
```

```
input [7:0] temp_a, temp_c;
```

```
output [7:0] temp_b, temp_d;
```

```
reg [7:0] temp_b, temp_d;
```

```
`include "mytask.v"
```

```
always @ (temp_a)
```

```
begin
```

```
    convert (temp_a, temp_b);
```

```
end
```

```
always @ (temp_c)
```

```
begin
```

```
    convert (temp_c, temp_d);
```

```
end
```

```
endmodule
```

```
task convert;
```

```
input [7:0] temp_in;
```

```
output [7:0] temp_out;
```

```
begin
```

```
    temp_out = (9/5) * (temp_in + 32)
```

```
end
```

```
endtask
```

Task is written in a separate file → it can be used in multiple modules

Source: http://www.asic-world.com/verilog/task_func1.html#Task

Task example 2 -

Calling a task coded in a separate file

```
module bus_wr_rd_task();  
reg clk,rd,wr,ce;  
reg [7:0] addr,data_wr,data_rd;  
reg [7:0] read_data;  
//-----  
`include "cpu_write.v"  
`include "cpu_read.v"  
  
//Instance DUT module  
mem_model ram_256 (  
    .data_rd (data_rd,  
    .data_wr (data_wr),  
    .addr (addr),  
    .rd (rd),  
    .wr (wr),  
    .ce (ce)  
);
```

```
//-----  
initial begin  
    clk = 0;  
    read_data = 0;  
    rd = 0;  
    wr = 0;  
    ce = 0;  
    addr = 0;  
    data_wr = 0;  
    data_rd = 0;
```

```
// Call the write and read tasks here  
#1 cpu_write(8'h11,8'hAA);  
#1 cpu_read(8'h11,read_data);  
#1 cpu_write(8'h12,8'hAB);  
#1 cpu_read(8'h12,read_data);  
#1 cpu_write(8'h13,8'h0A);  
#1 cpu_read(8'h13,read_data);  
#100 $finish;  
end  
  
// Clock Generator  
always  
    #1 clk = ~clk;  
  
endmodule
```

Source: http://www.asic-world.com/verilog/task_func1.html#Task

Task example 2 (cont.) -

CPU Write Task file (separate file)

```
task cpu_write;
input [7:0] address;
input [7:0] data;
begin
    $display ("%g CPU Write task with address : %h Data : %h",
        $time, address, data);
    $display ("%g -> Driving CE, WR, WR data and ADDRESS on to bus",
        $time);
    @ (posedge clk);
    addr = address;
    ce = 1;
    wr = 1;
    data_wr = data;
    @ (posedge clk);
    addr = 0;
    ce = 0;
    wr = 0;
    $display ("=====");
end
endtask
```

Source: http://www.asic-world.com/verilog/task_func1.html#Task

Task example 2 (cont.) -

CPU Read Task file (separate file)

```
task cpu_read;
input [7:0] address;
output [7:0] data;
begin
    $display ("%g CPU Read task with address : %h", $time, address);
    $display ("%g -> Driving CE, RD and ADDRESS on to bus", $time);
    @ (posedge clk);
    addr = address;
    ce = 1;
    rd = 1;
    @ (negedge clk);
    data = data_rd;
    @ (posedge clk);
    addr = 0;
    ce = 0;
    rd = 0;
    $display ("%g CPU Read data      : %h", $time, data);
    $display ("=====");
end
endtask
```

Source: http://www.asic-world.com/verilog/task_func1.html#Task

Task example 2 (cont.) -

Module memory model (DUT)

```
module mem_model (data_rd, data_wr, addr, rd, wr, ce);  
output reg [7:0] data_rd;  
input [7:0] data_wr;  
input [7:0] addr;  
input rd, wr, ce;  
//=====  
reg [7:0] mem [0:255];  
  
always @ (addr or ce or rd or wr or data_wr)  
if (ce) begin  
    if (wr) begin  
        mem[addr] = data_wr;  
    end  
    if (rd) begin  
        data_rd = mem[addr];  
    end  
end  
endmodule
```

Source: http://www.asic-world.com/verilog/task_func1.html#Task

Disabling tasks

```
task proc_a;  
begin  
...  
...  
    if(a == 0)  
        disable proc_a; // return if true  
...  
...  
end  
endtask
```

Difference

- **Functions**

- Can enable (call) just another function (not task)
- Execute in 0 simulation time
- No timing control statements allowed
- At least one input
- Return only a single value

- **Tasks**

- Can enable other tasks and functions
- May execute in non-zero simulation time
- May contain any timing control statements
- May have arbitrary input, output, or inout
- Do not return any value

Difference

- Tasks can be used for almost any common Verilog code
- Functions are used when the common code is purely combinational
- Functions are typically used for conversions and commonly used calculations

Sameness

- Both of **Task** and **Function**:
 - are defined in a module
 - are local to the module
 - can have local variables (registers, but not nets)
 - contain only procedural assignment
 - do **not** contain initial or always statements
 - are called from initial or always statements or other tasks

System task & function

- Appear in form: **\$<keyword>**
- They are considered part of the Verilog HDL. These system tasks and functions are divided into some categories as follows:
 - **Display tasks** : \$display, \$monitor, \$strobe, \$writ, \$dumpfile, \$dumpvars...
 - **File I/O tasks** : \$fclose, \$fdisplay, \$swrite, \$fread, \$sdf_annotate, \$readmemb, \$readmemh...
 - **Simulation control tasks**: \$finish, \$stop
 - **Math functions**: \$ln, \$log10, \$exp, \$sqrt, \$sin, \$cos, \$asin, \$acos...

(many more in Verilog IEEE standard...)

Ex: \$display ("Hello world");

System task & function

- \$time - returns the current simulation time
- \$display - displaying on screen
- \$stop - stops simulation
- \$finish - ends simulation
- \$monitor –monitor signal values, display signal wherever they change value

...

Ex: always @ (posedge clk)

\$display (“data is %h at time %t”, data, \$time);

Summary

- How to define **Task** and **Function**
- Where to use each of them
 - The same purpose as subroutines in software programming
 - Provide more readability, easier code management
 - Parts of design hierarchy in Behavioral Modeling
 - Tasks are more general than functions
 - is used for both combinational and sequential logic
 - Functions can only model purely combinational calculations
- System tasks and functions